

Now, let's explore the concept of clustering and similarity, and try to find out the documents of our interest. Let's assume that we want to read an article on soccer. We like an article and would like to retrieve another article that we may be interested in reading. The question is how do we do this? In the market, there are lots and lots of articles that we may or may not be interested in. We have to think of a mechanism that suggests articles that interest us. One of the ways is to have a word count of the article, and suggest articles that have the highest number of similar words. But there is a problem with this model as the document length can be excessive, and other unrelated documents can also be fetched as they might have many similar words. For example, articles on football players' lives may also get suggested, which we are not interested in. To solve this, the TF-IDF model comes in. In this model, the words are prioritised to find the related articles.

Let's get hands-on for the document retrieval. The first thing you need to do is to install GraphLab Create, on which Python commands can be run. GraphLab Create can be downloaded from <https://turi.com/> by filling in a simple form, which asks for a few details such as your name, email id, etc. GraphLab Create has the IPython notebook, which is used to write the Python commands. The IPython notebook is similar to any other notebook with the advantage that it can display the graphs on its console.

Open the IPython notebook which runs in the browser at <http://localhost:8888/>. Import GraphLab using the Python command:

```
import graphlab
```

Next, import the data in Sframe using the following command:

```
peoples = graphlab.SFrame('people_wiki.gl/')
```

To view the data, use the command:

```
peoples.head()
```

This displays the top few rows in the console.

The details of the data are the URL, the name of the person and the text from Wikipedia.

I will now list some of the Python commands that can be used to search for related articles on US ex-President Barack Obama.

1. To explore the entry for Obama, use the command:

```
obama = people[people['name'] == 'Barack Obama']
```

2. Now, sort the word counts for the Obama article. To turn the dictionary of word counts into a table, give the

Figure 1 shows a Jupyter notebook interface with the title 'Assignment Week 4'. The code cell contains `peoples.head()`, and the output displays a table with three columns: 'URL', 'name', and 'text'. The first row is for Digby Murrell, and the last row is for Trevor Ferguson.

URL	name	text
<http://idpedia.org/news/roce/Digby_Murrell>	Digby Murrell	digby murrell born 10 october 1979 is a former
<http://idpedia.org/news/roce/Alfred_J_Levy>	Alfred J. Levy	alfred j levy aka sandy levy graduated from ...
<http://idpedia.org/news/roce/Harold_Brown>	Harold Brown	harold brown is a singer and harmonica player who
<http://idpedia.org/news/roce/Franz_Rottensteiner>	Franz Rottensteiner	franz rottensteiner born in waldmannsfeld lower ...
<http://idpedia.org/news/roce/O-Erika>	O-Erika	henry kreutz born 30 december 1974 in tallinn
<http://idpedia.org/news/roce/Sam_Henderson>	Sam Henderson	sam henderson born october 18 1966 is an
<http://idpedia.org/news/roce/Aaron-LaCrate>	Aaron LaCrate	aaron lacrate is an american music producer
<http://idpedia.org/news/roce/Trevor_Ferguson>	Trevor Ferguson	trevor ferguson aka john ferroz born 11 november

Figure 1: The people data loaded in Sframes

Figure 2 shows a Jupyter notebook with the title 'Exploring the entry for president Obama'. The code cell contains `obama = peoples[peoples['name'] == 'Barack Obama']`, and the output displays a table with three columns: 'URL', 'name', and 'text'. The first row is for Barack Obama.

URL	name	text
<http://idpedia.org/news/roce/Barack_Obama>	Barack Obama	barack hussein obama is an american born august 13 1961 is the 44th and current president of the united states and the first african american to hold the office born in kenya his father is a graduate of harvard university and earned his phd

Figure 2: Data generated for the Obama article

Figure 3 shows a Jupyter notebook with the title 'Sort the word counts for the Obama article'. The code cell contains `obama_word_count_table = obama[['word_count']].stack('word_count', new_column_name = ['word', 'count'])`, and the output displays a table with two columns: 'word' and 'count'. The first row is for 'normalized' with a count of 1.

word	count
normalized	1
sought	1
combat	1
continued	1
unconstitutional	1
is	1
california	1
1998	1
managing	1

Figure 3: Sorting the word count

following command:

```
obama_word_count_table = obama[['word_count']].stack('word_count', new_column_name = ['word', 'count'])
```

3. To sort the word counts to show the most common words at the top, type:

```
obama_word_count_table.head()
```